FRESCO: A Novel Consistency Control for Asynchronous Pipeline Parallel Training[1]

Namhoon Lee

POSTECH

October 2026

[1] a work of Dongyeop Lee and myself to be published at NeurIPS 2026

Guess this number?

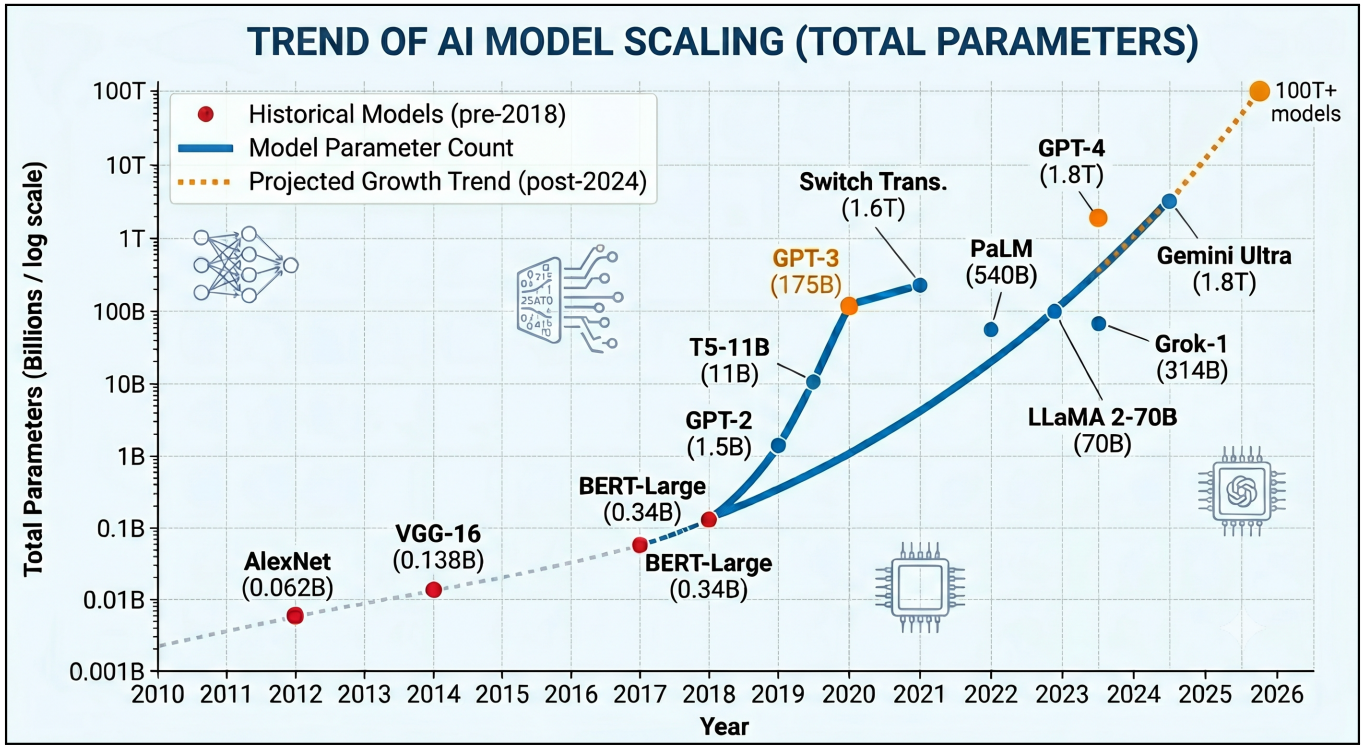
$$1.04 \times 10^{12}$$

Kimi K2



- Largest open-weight LLM series from Moonshot AI (alongside DeepSeek V4; much larger than Meta's Llama or Alibaba's Qwen)
- Massive MoE architecture containing 1 trillion parameters, supporting reasoning, long-horizon autonomous tasks, tool-calling, etc. handling complex data or coding workflows

Scaling trend



- frontier models are scaling to trillions of total parameters
- compute-per-token is scaling down (ultra-sparse MoEs), shifting to inference-time scaling and scaling via parallel agent swarms

Scaling law

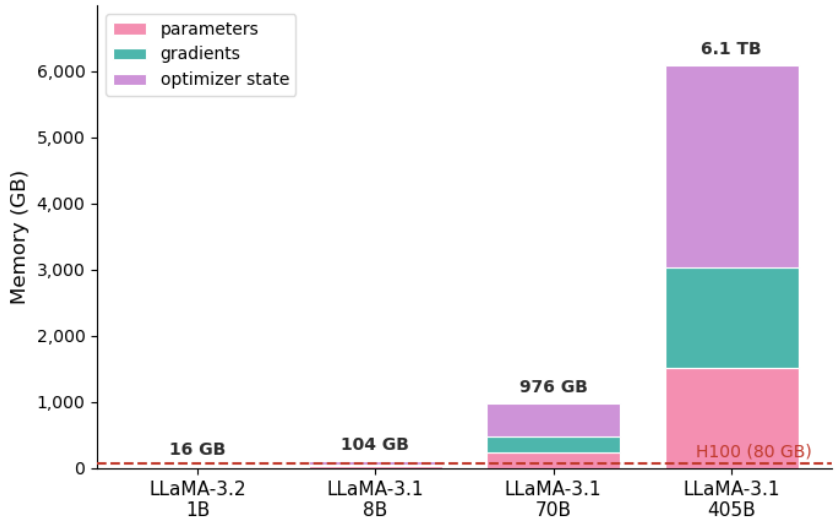
"Chinchilla scaling law" by Hoffman et al. (2022):

$$L(N, D) = \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} + L_{\infty}$$

N number of parameters, D number of training tokens

- Model performance improves with parameters, data, and compute.
- Parameters and training tokens should scale in equal proportion.
- The two power-law terms show that loss decreases with more parameters or data, but with diminishing returns on each.

The memory wall: models no longer fit on a single device



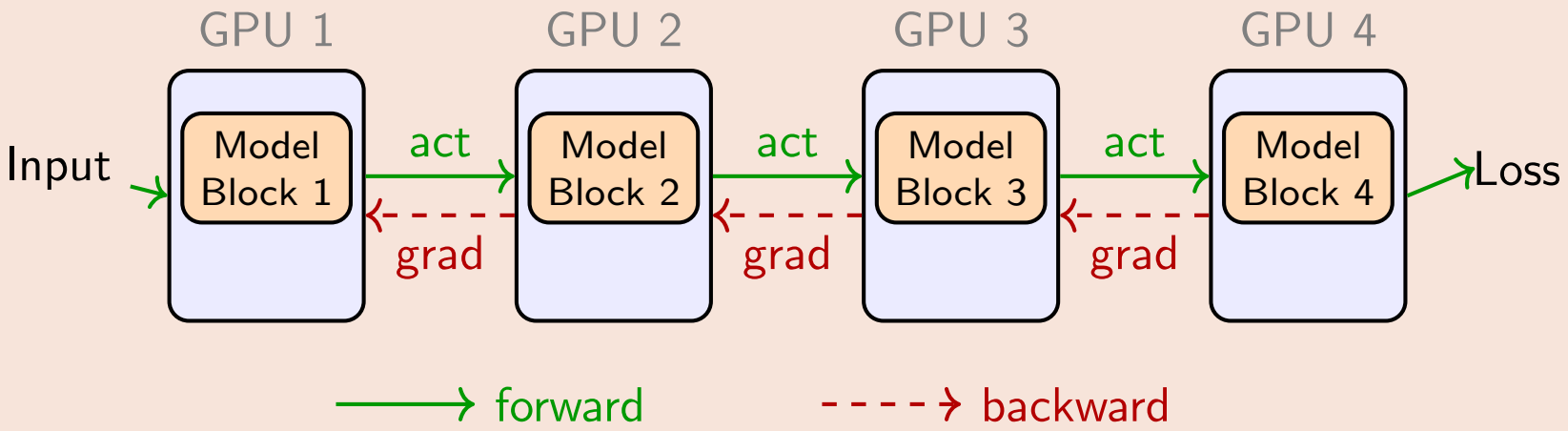
Memory requirement by model size

- Even an 8B model requires 104 GB — already exceeds one H100 (80 GB).
- A 70B model requires 976 GB — $12\times$ a single GPU.

$\Rightarrow$ Multi-device training is unavoidable.

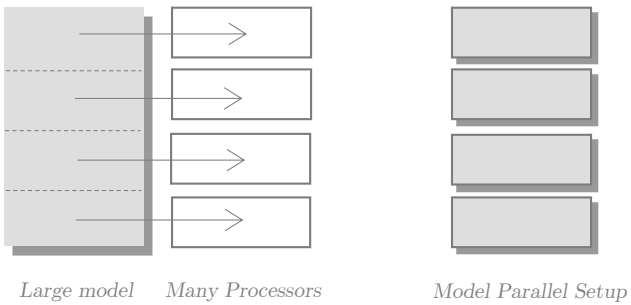
Training large models on a single device is physically impossible. The model *must* be distributed across multiple accelerators.

Pipeline parallelism: splitting the model across devices

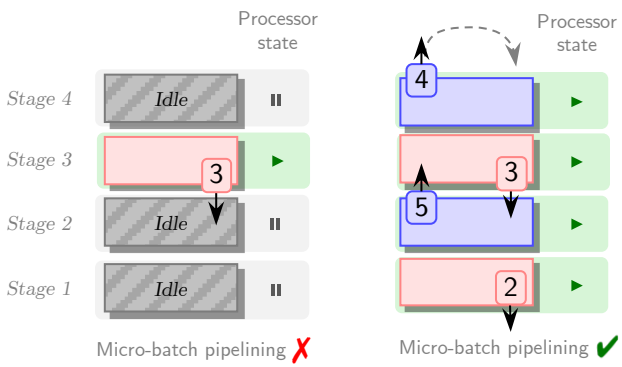


- Split layers into sequential *stages*
- Each stage lives on a separate device
- Only activations/activation gradients passed between stages

Pipelining for fully-utilized model-distributed training

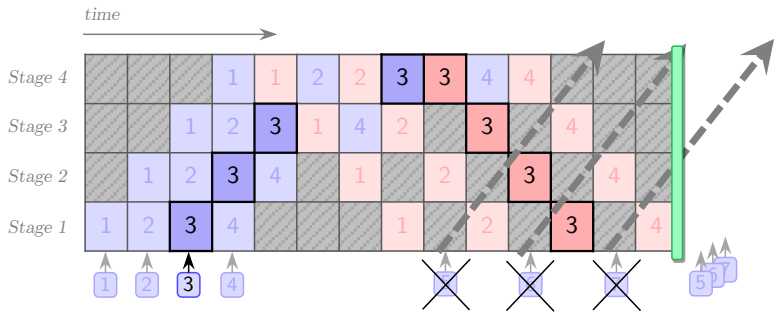


- Model scaling necessitates distributing its shard across multiple GPUs
- This is *pipeline parallelism* (PP)



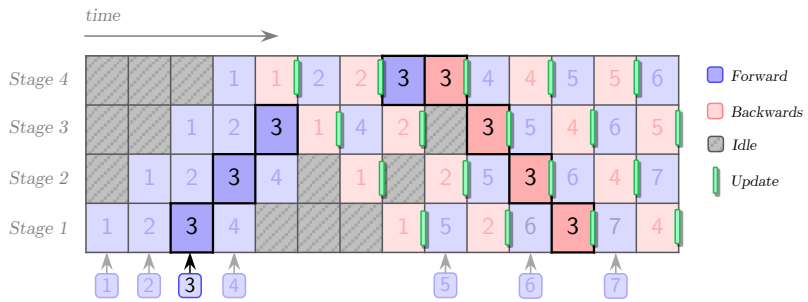
- Processors run at all times with **micro-batch pipelining**
- This increases throughput for gradient computation

AsyncPP for advancing throughput



SyncPP: periodic halts

- Sound gradient
- but throughput ↓

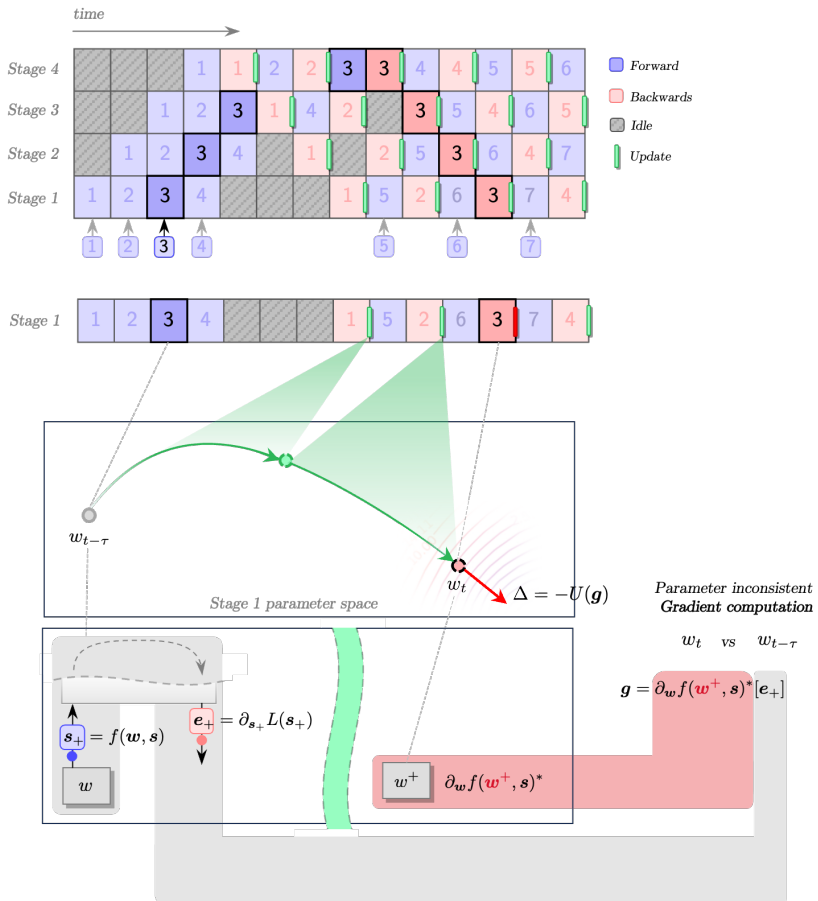


AsyncPP: halt-less gradient compute

- throughput ↑
- but degenerate gradient

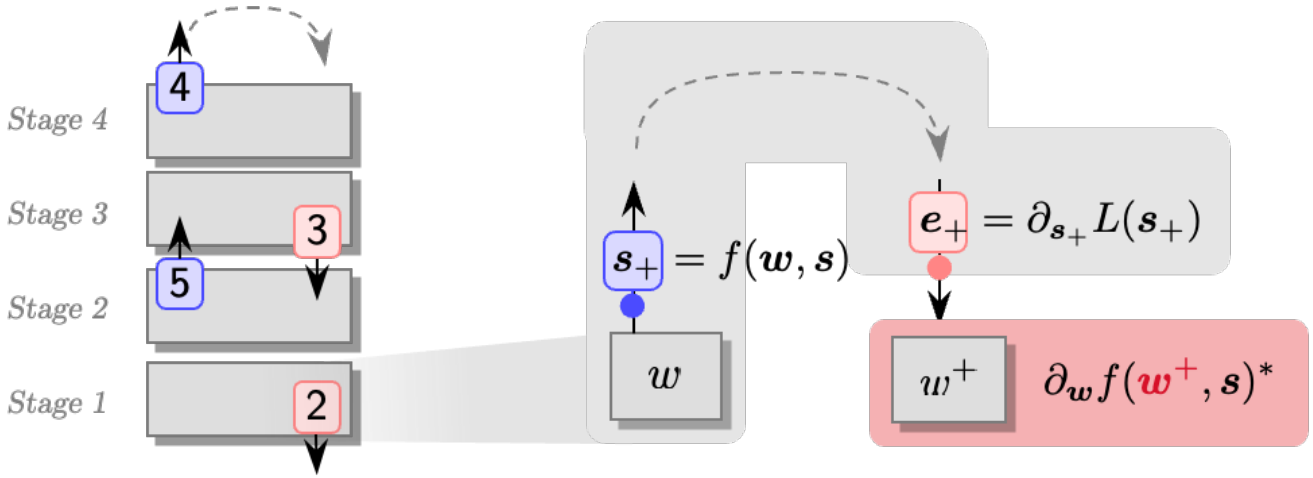
AsyncPP: Can we develop optimization-side remedy to preserve benefits of high throughput?

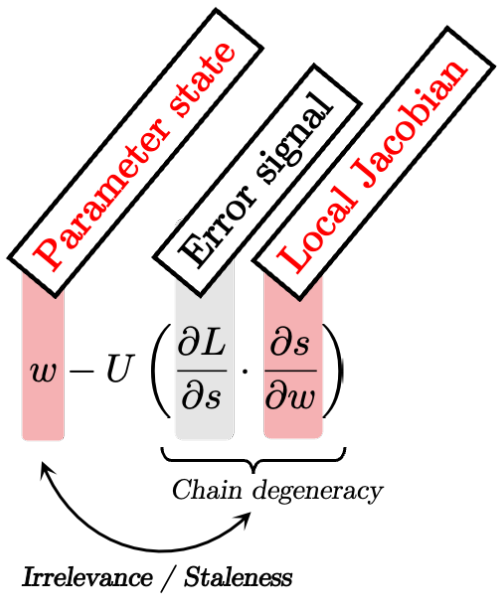
Close look: Asynchronous parameter drifting problem



- Different parameters at forward-/backward-time during one gradient computation
- Parameter evolution abandons forward-time computations
- They become stale for updating **the current parameter**

Drift causes gradient chain degeneracy and irrelevance





- Gradient may not belong to any parameter state due to partial gradient mismatch
- or the computed gradient may be stale even when it is "sound"

Prior AsyncPP approaches (slightly) miss the point

$$w - U\left(\frac{\partial L}{\partial s} \cdot \frac{\partial s}{\partial w}\right)$$

Retrospective parameter equivalence

$$w - U\left(\frac{\partial L}{\partial s} \cdot \frac{\partial s}{\partial w}\right)$$

Prospective parameter equivalence

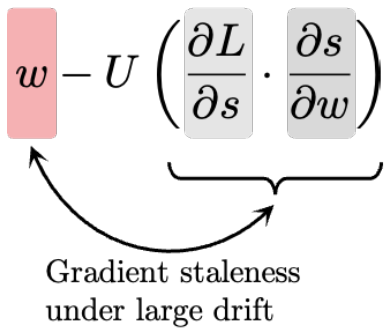
Parameter state | Error signal | Local Jacobian

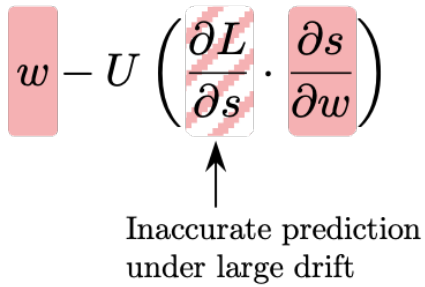
		w	$\frac{\partial L}{\partial s}$	$\frac{\partial s}{\partial w}$	downsides
Inconsistent		w_t	$w_{t-\tau}$	w_t	Degenerate gradients, irrelevant to **drift**ed weights
Retrospective	Stashing	w_t	$w_{t-\tau}$	$w_{t-\tau}$	Irrelevant to **drift**ed weights, need high stash memory (all prior τ checkpoints)
	Recovery	w_t	$w_{t-\tau}$	$\hat{w}_{t-\tau}$	Irrelevant to **drift**ed weights, prediction error/instability
Prospective	Prediction	w_t	$\hat{w}_t$	w_t	prediction error/instability with larger **drift**

Parameter inconsistent Gradient chain

Irrelevance to drifted parameter state

Parameter drift is uncontrolled[2]

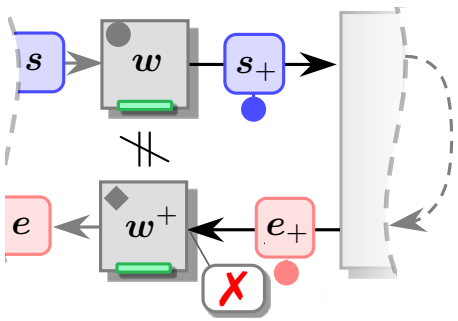




- they are not directly addressing the invasive parameter drifts
- the source of the gradient degeneracy is left uncontrolled

[2] not to mention, stashing requires a tremendous amount of memory

It is not obvious on how to control this drift



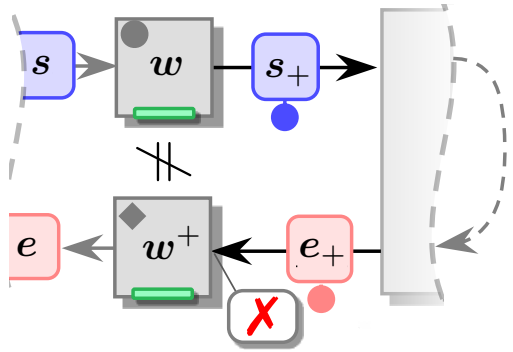
- too small updates: waste of obtained gradient information and progress
- too large updates: degrades concurrent gradient compute

What would be a minimal condition for asynchornous updates to not degrade concurrent gradient computes?

We need a principle to better characterize the interaction

What is the minimal mathematical sacrifice required of the current update to avoid degrading concurrent gradient computation?

A closer look at the gradient computation



$$\boldsymbol{s}_+ = f(\boldsymbol{w}, \boldsymbol{s}) \qquad \triangleright \textit{ Forward signal}$$

$$(w \to \boldsymbol{w}^+, \text{ wait for } \partial_{\boldsymbol{s}_+} L) \qquad \triangleright \textit{ Asynchronous updates}$$

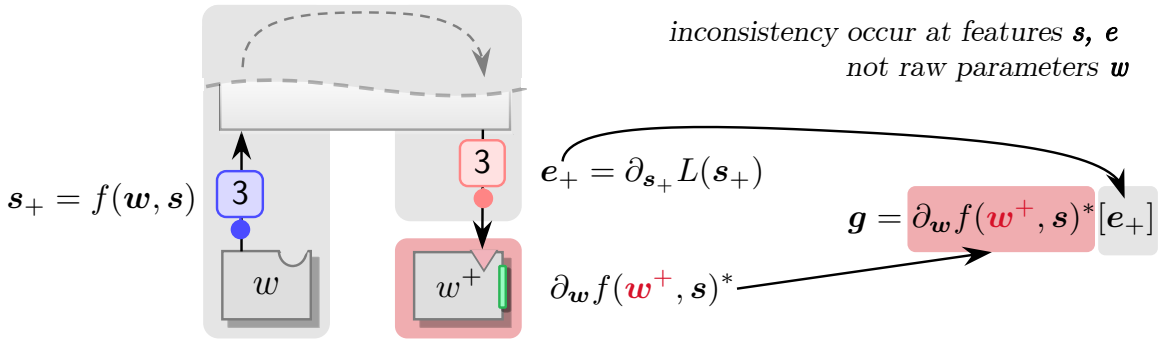
$$\boldsymbol{e}_+ = \partial_{\boldsymbol{s}_+} L(\boldsymbol{s}_+) \qquad \triangleright \textit{ Returned error signal}$$

$$\boldsymbol{e} = \partial_{\boldsymbol{s}} f(\boldsymbol{w}^+, \boldsymbol{s})^*[\boldsymbol{e}_+] \qquad \triangleright \textit{ Backprop error signal}$$

$$\boldsymbol{g} = \partial_{\boldsymbol{w}} f(\boldsymbol{w}^+, \boldsymbol{s})^*[\boldsymbol{e}_+] \qquad \triangleright \textit{ Compute gradient}$$

$$\Delta = -U(\boldsymbol{g}) \qquad \triangleright \textit{ Compute update}$$

e_+ depends on s_+, not directly on w



$s_+ = f(w, s)$ ▷ *Forward signal*

$(w \to w^+,$ wait for $\partial_{s_+} L)$ ▷ *Asynchronous updates*

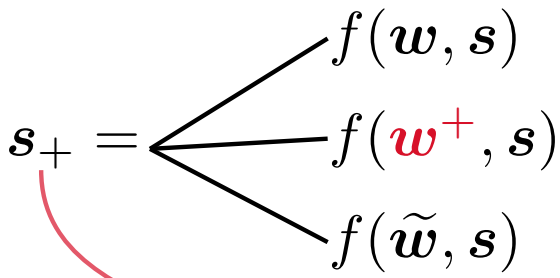
$e_+ = \partial_{s_+} L(s_+)$ ▷ *Returned error signal*

$e = \partial_s f(w^+, s)^*[e_+]$ ▷ *Backprop error signal*

$g = \partial_w f(w^+, s)^*[e_+]$ ▷ *Compute gradient*

$\Delta = -U(g)$ ▷ *Compute update*

Forward-inconsistency



▷ *Possible forward states*

$(w \to w^+,$ wait for $\partial_{s_+} L)$ ▷ *Asynchronous updates*

$e_+ = \partial_{s_+} L(s_+)$ ▷ *Returned error signal*

$e = \partial_s f(w^+, s)^*[e_+]$ ▷ *Backprop error signal*

$g = \partial_w f(w^+, s)^*[e_+]$ ▷ *Compute gradient*

$\Delta = -U(g)$ ▷ *Compute update*

Any $w, w^+, \tilde{w}$ with the same s_+ will have, thus can share, the same e_+.

Forward-equivalence principle

Gradient soundness under forward equivalence

Let $\boldsymbol{s}_+ = f(\boldsymbol{w}, \boldsymbol{s})$ and let $\boldsymbol{e}_+ = \partial_{\boldsymbol{s}_+} L(\boldsymbol{s}_+)$ be the returned error signal computed under $\boldsymbol{w}$. For any updated parameter $\boldsymbol{w}^+$ satisfying

$$f(\boldsymbol{w}^+, \boldsymbol{s}) = f(\boldsymbol{w}, \boldsymbol{s}) = \boldsymbol{s}_+,$$

the adjoint pullback

$$\boldsymbol{g} = \boldsymbol{\partial}_{\boldsymbol{w}} f(\boldsymbol{w}^+, \boldsymbol{s})^* [\boldsymbol{e}_+]$$

is a valid gradient of the same loss evaluated at $\boldsymbol{w}^+$.

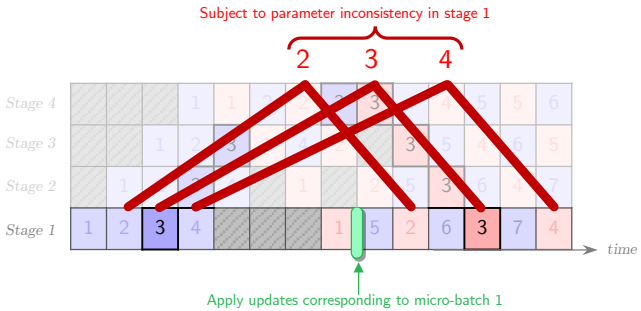
Forward-Equivalent Subspace-Constrained Optimization

The idea of FRESCO is to allow $\boldsymbol{w}$ to change freely under its subspace that keeps forward-equivalence:

$$f(\boldsymbol{w}^{+}, \boldsymbol{s}) = f(\boldsymbol{w}, \boldsymbol{s})$$

for affected micro-batches.

Setup



- We can perform update $W^+ \leftarrow W + \Delta$ for finished `MB1`,
- which will affect the `MB2-4` with layer input $\bar{X}$

$$
\begin{aligned}
Y &= W\bar{X} \\
&\parallel \text{2.} \\
Y &= (W + \Delta')\bar{X} \\
&\approx \text{1.} \\
&\Delta
\end{aligned}
$$

For every layer, the goal is to construct Δ' that

1. is similar to the original Δ, and
2. preserves forward-equivalence for $\bar{X}$

Problem formulation

$$\min_{\Delta'} \frac{1}{2}\|\Delta' - \Delta\|_F^2$$

$$\text{s.t. } (W + \Delta')\bar{X} = W\bar{X}$$

$$\text{s.t. } \Delta'\bar{X} = 0$$

- We want to make a minimal adjustment from $\Delta \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ that
- Preserves forward on protected $\bar{X}$
- $\bar{X} \in \mathbb{R}^{d_{\text{in}} \times \tau n}$: in-flight inputs

- Don't need raw W for this
- Remove component of Δ that affects $\bar{X}$

Solving with KKT conditions

$$\begin{aligned} \min_{\Delta'} \quad & \tfrac{1}{2}\|\Delta' - \Delta\|_F^2 \\ \text{subject to} \quad & \Delta' \bar{X} = 0 \end{aligned}$$

KKT is necessary and sufficient for convex problems:

$$\begin{aligned} \nabla_{\Delta'} \left(\tfrac{1}{2}\|\Delta' - \Delta\|_F^2 + \langle \Lambda, \Delta' \bar{X} \rangle_F \right) &= 0 \quad \text{(Lagrangian stationarity)} \\ \Delta' \bar{X} &= 0 \quad \text{(primal feasibility)} \\ \text{no inequality constraint} \quad & \text{(complementary slackness)} \\ \text{no inequality constraint} \quad & \text{(dual feasibility)} \end{aligned}$$

Two equations with two variables: primal Δ' and dual Λ.

Solving for Λ via Moore–Penrose

$$(\Delta' - \Delta) + \Lambda \bar{X}^\top = 0$$

$$\Delta' = \Delta - \Lambda \bar{X}^\top \qquad (1)$$

$$\Delta' \bar{X} = 0$$

$$(\Delta - \Lambda \bar{X}^\top) \bar{X} = 0$$

$$\Lambda \bar{X}^\top \bar{X} = \Delta \bar{X}$$

$$\Lambda = \Delta \bar{X} \, (\bar{X}^\top \bar{X})^\dagger$$

- Manipulate stationarity condition
- Substitute Δ' with $\Delta - \Lambda \bar{X}^\top$
- We use the Moore–Penrose pseudoinverse since $\bar{X}^\top \bar{X}$ may be singular

Final solution

$$\begin{aligned}\Delta' &= \Delta - \Lambda \bar{X}^\top \\ &= \Delta - \Delta \bar{X} \, (\bar{X}^\top \bar{X})^\dagger \bar{X}^\top\end{aligned}$$

- Substituting back to (1) gives the solution

$$\Delta_\star = \Delta \left(I - \bar{X} (\bar{X}^\top \bar{X})^\dagger \bar{X}^\top \right)$$

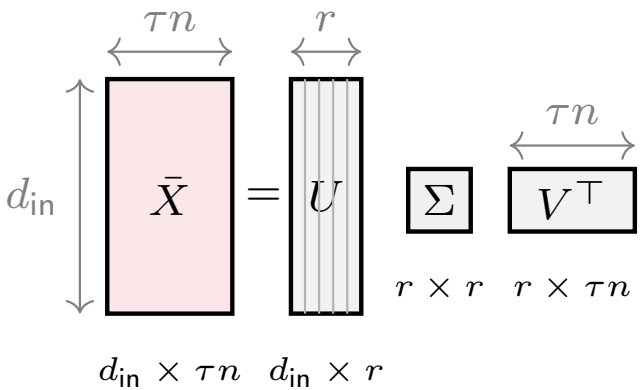
Singular basis form of FRESCO

$$\Delta_\star = \Delta(I - \underbrace{\bar{X}(\bar{X}^\top \bar{X})^\dagger \bar{X}^\top})$$

$$\begin{aligned}
\bar{X} &= U\Sigma V^\top \\
(\bar{X}^\top \bar{X})^\dagger &= V\Sigma^{-2}V^\top \\
\bar{X}(\bar{X}^\top \bar{X})^\dagger \bar{X}^\top &= U\Sigma V^\top V\Sigma^{-2}V^\top V\Sigma U^\top \\
&= U\Sigma\Sigma^{-2}\Sigma U^\top \\
&= UU^\top
\end{aligned}$$

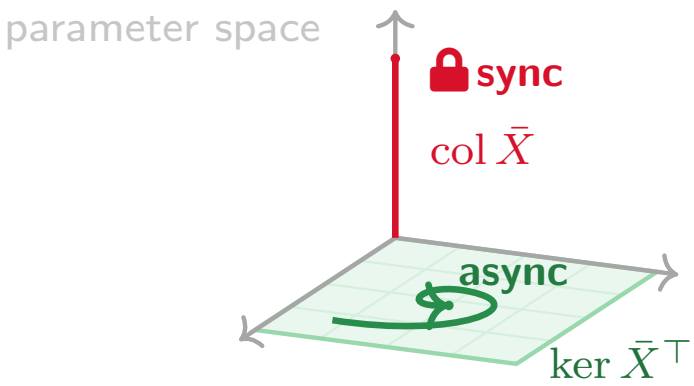
$$\Delta_\star = \Delta(I - \overbrace{UU^\top})$$

FRESCO = lock $\operatorname{col} \bar{X}$ and async update over $\ker \bar{X}^\top$



$$\operatorname{col}(\bar{X}) = \operatorname{span}(U),$$
$$UU^\top : \text{ projection onto } \operatorname{col} \bar{X},$$
$$I - UU^\top : \text{ projection onto } \ker \bar{X}^\top.$$

$$\Delta_\star = \operatorname{Proj}_{\ker \bar{X}^\top}(\Delta)$$



- Halt/update is performed at the parameter-subspace level
- sync and async are somewhat simultaneously happening at each $\operatorname{col}(\bar{X})$ and $\ker(\bar{X}^\top)$ (semi-sync?)

When pipeline gets extremely deep

$$\tau \uparrow \implies \operatorname{rank}(\bar{X}) \uparrow$$

$$\operatorname{col}(\Delta^\top) \perp \ker(\bar{X}) \iff \Delta_\star = 0$$

- More in-flight micro-batches
- Fewer free directions for updates

$$\arg\min_{\operatorname{rank}(A)=k} \|\bar{X} - A\|_F = U_k \Sigma_k V_k^\top$$

$$\Delta_\star = \Delta(I - U_k U_k^\top)$$

- We protect the spectrally significant rank-k of $\bar{X}$
- Eckart–Young–Mirsky theorem

$$k = \min\left(\tau n, \lceil \alpha \min(d_{\text{out}}, d_{\text{in}}) \rceil\right)$$

- Single hyperparameter $\alpha \in [0, 1]$
- Per-layer adaptive

Forward enqueues, backward dequeues and projects

Algorithm FRESCO step (per linear layer)

Require: W, X, G, α, mode
State: FIFO queue $\mathcal{Q}$ of low-rank bases
if *mode* $=$ FWD **then**
  $k \leftarrow \lceil \alpha \min(d, n) \rceil$
  $U_k \leftarrow \textsc{ExtractBasis}(X, k)$
  $\mathcal{Q}.\textsc{enqueue}(U_k)$
end if
if *mode* $=$ BWD **then**
  $\Delta \leftarrow \textsc{OptUpd}(W, G)$
  $\mathcal{Q}.\textsc{dequeue}()$
  $Q \leftarrow \textsc{Deduplicate}(\mathcal{Q})$
  $W \leftarrow W + \Delta(I - QQ^\top)$
end if

- FIFO of low-rank activation bases
- Forward enqueues a fresh basis
- Backward dequeues, then projects Δ
- ExtractBasis: 3 steps block power + Newton–Schulz
- Deduplicate: stack and orthogonalize

How do we analyze the convergence of AsyncPP?

$$\widetilde{\boldsymbol{g}}_t = \nabla \mathcal{L}(\boldsymbol{w}_t) + \underbrace{\xi_t}_{\text{SGD noise}} + \underbrace{\delta_t}_{\text{async bias}}$$

SyncPP

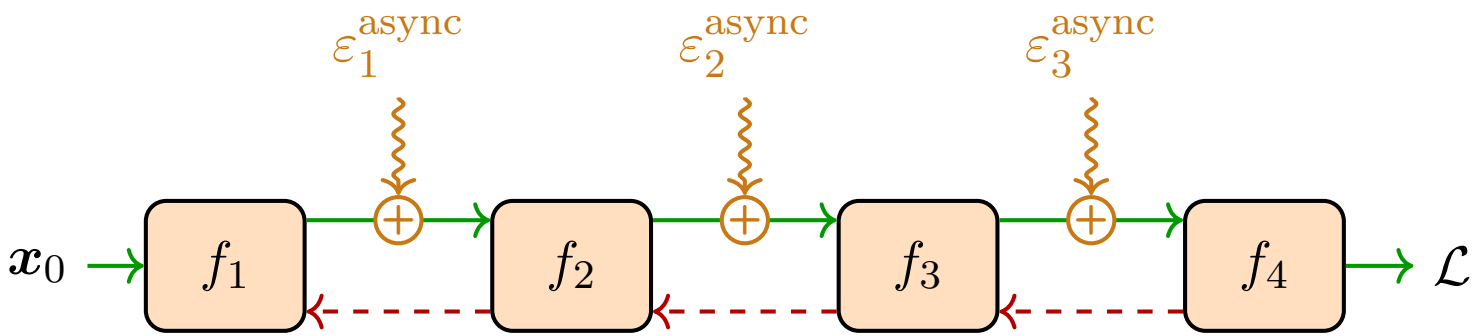
$\delta_t = 0$ ✓

AsyncPP

$\delta_t \neq 0$ ×

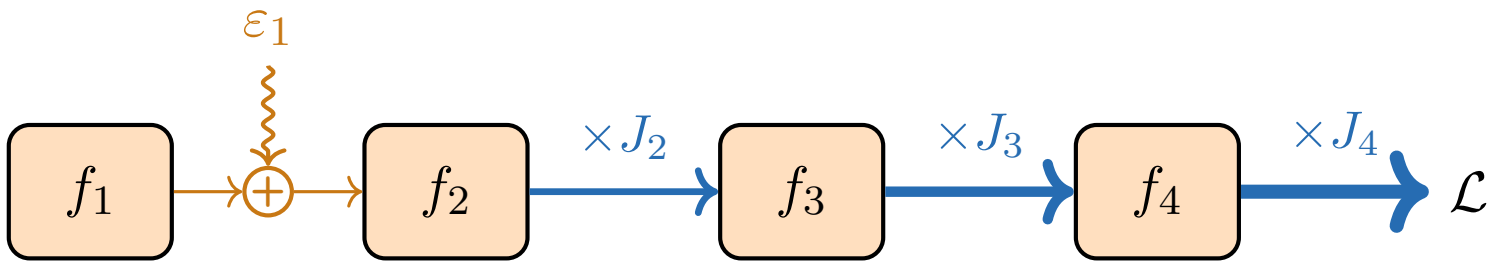
Goal: bound the whole-model bias δ_t by what each *stage* does

Model AsyncPP as SGD with perturbed forward



Model: backward runs on current weights; only the forward is stale, leaving a *local* perturbation $\varepsilon_i^{\text{async}}$ from each stage's update Δ_i (Kong et al. 2026)

Local perturbations amplify through depth



$$\|\delta_t\| \leq \underbrace{\Gamma_N}_{\substack{\text{depth amp.} \\ \mathcal{O}(e^{cN})}} \sum_i \varepsilon_i^{\text{async}}$$

Lever: Γ_N is fixed by depth, so shrink each local $\varepsilon_i^{\text{async}}$

Rank-k FRESCO leaves only the spectral tail

$$\bar{X}_i = \underbrace{[\bar{X}_i]_k}_{\text{top-}k\text{ SVD}} + \underbrace{\bar{X}_i^{\text{tail}}}_{\text{rest}}, \qquad \Delta_i' = \Delta_i(I - U_k U_k^\top)$$

AsyncPP

$$\varepsilon_i^{\text{async}} = \|\Delta_i \bar{X}_i\|$$

FRESCO-k

$$\varepsilon_i^{\text{async}} = \|\Delta_i \bar{X}_i^{\text{tail}}\|$$

Effect: top-k directions are protected; only the tail perturbs the forward

Convergence of rank-k FRESCO

Theorem ($\eta \le 1/(4L)$)

$$\frac{1}{T}\sum_t \mathbb{E}\|\nabla \mathcal{L}(\boldsymbol{w}_t)\|^2 \le \underbrace{\frac{2(\mathcal{L}(\boldsymbol{w}_0)-\mathcal{L}_\star)}{\eta T} + L\eta\sigma^2}_{\text{standard SGD}} + \underbrace{\frac{5\Gamma_N^2}{2T}\sum_t \mathbb{E}\Big(\sum_i \varepsilon_{i,t}^{\text{async}}\Big)^2}_{\text{async bias } \|\delta_t\|^2}$$

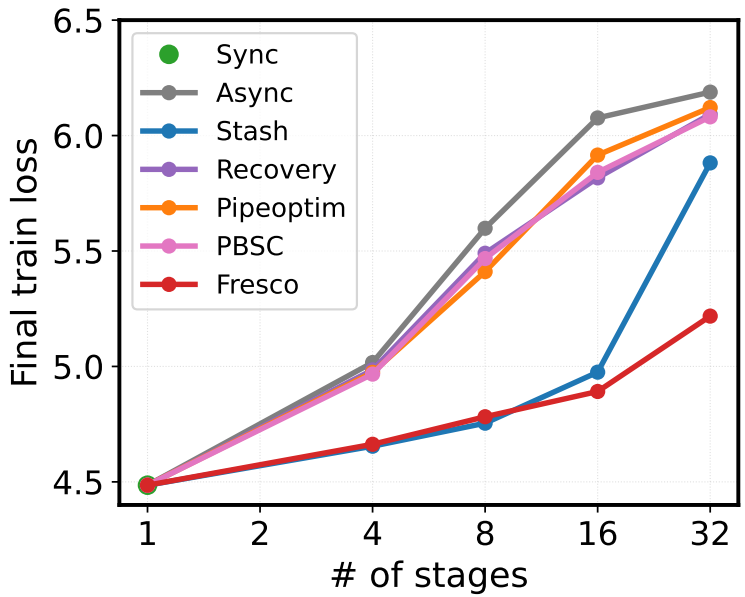
$$\varepsilon_{i,t}^{\text{async}} \le \|\Delta_{i,t}\|_2 \, \|\bar{X}_{i,t}^{\text{tail}}\|_F$$

Result: the whole async bias shrinks to the spectral tail of $\bar{X}_i$

95M NanoGPT pre-training on OpenWebText, $P \in \{1, \ldots, 32\}$

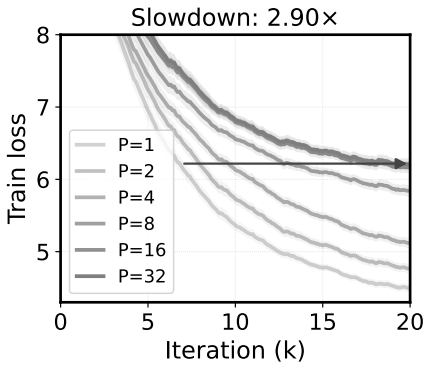
- **Model.** NanoGPT 95M decoder-only Transformer; OpenWebText, batch 8, seq 512, 20k iter.
- **Optimizer.** NAdam, $\beta_1 = 0.99$, $\texttt{MB} = 8$, Pipemare-lr, gradient clipping=1
- **Depths.** $P \in \{1, 2, 4, 8, 16, 32\}$ pipeline stages.
- **Baselines.** Async, Stash (Narayanan et al. 2019), Recovery (Yang et al. 2021), PipeOptim, PBSC (Guan et al. 2025; Kosson et al. 2021).
- **Metric.** Slowdown — iterations to target loss at P=32 vs. P=1.

FRESCO holds at $1.89\times$ slowdown; baselines degrade to 2.67–$2.90\times$

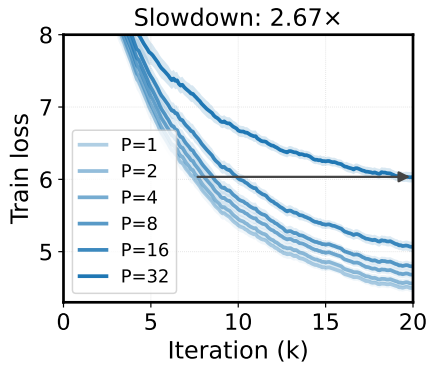


method	slowdown at P=32
Async	$2.90\times$
Recovery	$2.86\times$
PipeOptim	$2.88\times$
PBSC	$2.89\times$
Stashing	$2.67\times$
Fresco	$1.89\times$

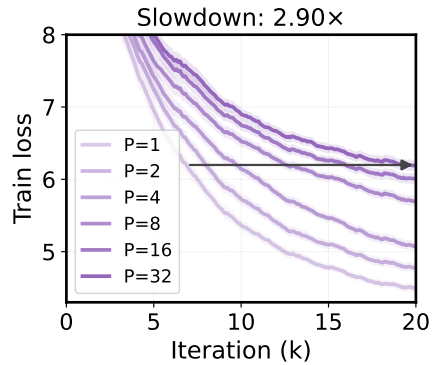
FRESCO curves collapse across depths; baselines spread



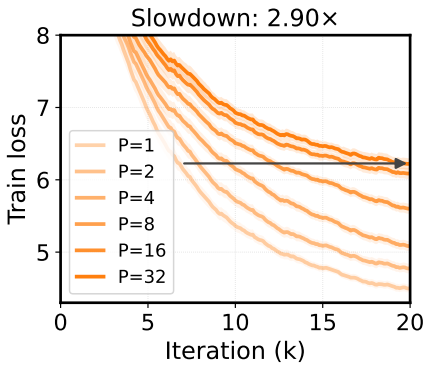
Async



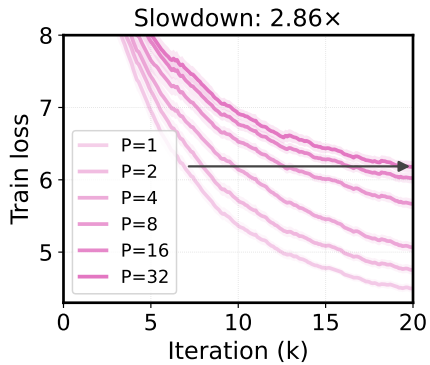
Stash



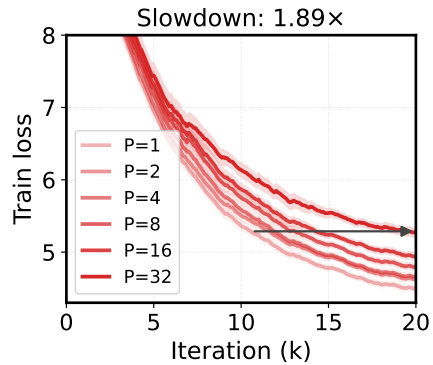
Recovery



PipeOptim

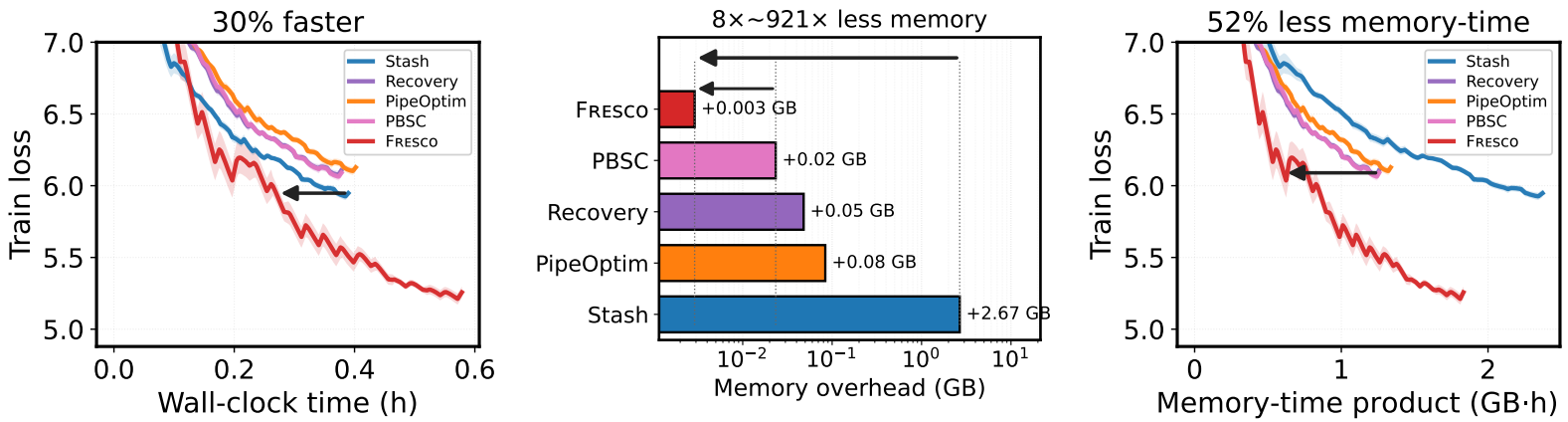


PBSC



FRESCO

FRESCO reaches the loss 30% faster with 921× less memory



(a) loss vs. GPU-hours (b) memory overhead (c) loss vs. memory

52% lower memory–time product, despite the extra projection compute.

FRESCO uses 921× less memory than parameter stashing

method	per-stage overhead
Fresco	**2.9 MB**
PBSC / Kosson LWP	23 MB
Parameter recovery (PipeMare)	48 MB
PipeOptim / SpecTrain	84 MB
Parameter stashing (PipeDream)	2,670 MB

95M NanoGPT, $P{=}32$, stage 0, $\tau{=}31$. No parameter copy, no prediction buffer.

Previews

- Beyond pure theory or throughput:

$$\text{Effective utility: } \frac{\text{loss}}{\text{time}} = \underbrace{\frac{\text{loss}}{\text{iter}}}_{\substack{\text{(theory)} \\ \text{convergence}}} \times \underbrace{\frac{\text{iter}}{\text{time}}}_{\substack{\text{(HW)} \\ \text{throughput} \\ \text{/utilization}}}$$

- Sync vs. Async?

Thank you

Questions?

References I

- Guan, Lei et al. (2025). "PipeOptim: Ensuring effective 1F1B schedule with optimizer-dependent weight prediction". In: *IEEE Transactions on Knowledge and Data Engineering*.
- Kong, Boao, Hengrui Zhang, and Kun Yuan (2026). "On the Convergence of Stochastic Gradient Descent with Perturbed Forward-Backward Passes". In: *arXiv*.
- Kosson, Atli et al. (2021). "Pipelined backpropagation at scale: training large models without batches". In: *MLSys*.
- Narayanan, Deepak et al. (2019). "PipeDream: Generalized pipeline parallelism for DNN training". In: *SOSP*.
- Yang, Bowen et al. (2021). "Pipemare: Asynchronous pipeline parallel dnn training". In: *MLSys*.